



Concorrência e Multithreading em Java

Autoria: Elder Moraes

Introdução

Nesta disciplina, mergulhamos profundamente no universo da concorrência na plataforma Java, com foco em preparar profissionais para os desafios da computação moderna. A concorrência é um dos pilares da engenharia de software contemporânea, especialmente em ambientes que demandam escalabilidade, desempenho e resposta eficiente a múltiplas requisições simultâneas.

Nosso objetivo foi capacitar você, aluno de pós-graduação com conhecimento prévio em Java, a dominar as ferramentas, estruturas e modelos de programação concorrente que a linguagem oferece. Desde os fundamentos do gerenciamento de threads até os recursos mais avançados como Virtual Threads e Structured Concurrency, esta disciplina foi estruturada para oferecer uma base sólida, ao mesmo tempo em que aponta para práticas modernas e eficientes.

Iniciamos explorando os conceitos essenciais: o que é concorrência, por que ela é importante, quais os principais desafios e quais paradigmas foram historicamente utilizados para lidar com tarefas simultâneas. Em seguida, abordamos ferramentas clássicas como o `ExecutorService`, as collections concorrentes, os synchronizers e o Fork/Join Framework, estabelecendo o domínio sobre os recursos que fazem parte do núcleo da linguagem desde versões anteriores.

A partir dessa base, avançamos para os recursos introduzidos nas versões mais recentes do Java. Estudamos o `CompletableFuture`, com sua abordagem funcional e assíncrona, e os `Parallel Streams`, que oferecem paralelismo declarativo. Por fim, exploramos as inovadoras Virtual Threads, uma revolução trazida pelo Project Loom, que promete conciliar legibilidade de código com escalabilidade extrema, redefinindo a forma como aplicações Java lidam com concorrência.

Ao longo das aulas, mantivemos uma abordagem teórica consistente, sempre acompanhada de exemplos práticos e exercícios que consolidam o aprendizado. Encerramos a disciplina com uma aula prática, integrando todos os conceitos em

exemplos completos, para que você possa ver como as ferramentas interagem e se complementam no desenvolvimento de aplicações reais.

Esta disciplina foi desenhada para que você não apenas compreenda os mecanismos de concorrência em Java, mas esteja preparado para tomar decisões arquiteturais conscientes, construir aplicações robustas e manter um alto padrão de qualidade em ambientes de produção.

Que este material sirva como uma referência contínua na sua carreira, apoiando o desenvolvimento de soluções modernas, performáticas e alinhadas com as melhores práticas da engenharia de software.

Capítulo 1: Overview

Neste primeiro capítulo, quero te dar uma visão geral sobre o que é programação concorrente e multithreading no contexto da linguagem Java. Esse é um assunto que, muitas vezes, passa despercebido por muitos desenvolvedores, mesmo

sendo algo que usamos constantemente, ainda que de forma indireta. A JVM e os frameworks que utilizamos no dia a dia já fazem muito trabalho relacionado a threads por nós. Contudo, entender como isso funciona por baixo dos panos é essencial para tomar boas decisões em ambientes corporativos.

Faço aqui um paralelo com estruturas de dados. Apesar de não manipulá-las diretamente com tanta frequência, entender como funcionam, suas características e como foram implementadas nos permite utilizar melhor as ferramentas que o Java nos oferece. O mesmo vale para concorrência e multithreading. Meu objetivo aqui é te preparar para entender os conceitos e fundamentos por trás das abstrações, para que você possa tomar decisões mais acertadas ao construir suas aplicações.

Concorrência não é um recurso "nice to have". Ela é uma necessidade em aplicações modernas. Primeiro, do ponto de vista de performance: é através da programação concorrente que conseguimos tirar proveito real dos múltiplos núcleos de processamento dos sistemas atuais. Isso resulta em aplicações mais responsivas e eficientes. Em segundo lugar, do ponto de vista de utilização de recursos: concorrência nos permite usar CPU, memória, disco e outros recursos de forma mais racional. Quanto mais próximo do uso ótimo desses recursos, melhor para a aplicação e para o ambiente como um todo.

Outro ponto é a escalabilidade. Ao crescer a base de usuários ou o volume de dados, saber usar concorrência é essencial para continuar entregando performance. A responsividade também melhora, permitindo que tarefas sejam executadas em segundo plano sem travar a experiência do usuário. E claro, sistemas em tempo real, como em plataformas financeiras ou jogos, só são viáveis com uma boa gestão concorrente.

Os conceitos básicos começam com a distinção entre processo e thread. Processo é uma unidade independente de execução, com seu próprio espaço de memória. Já a thread é uma unidade de execução dentro de um processo, compartilhando memória e recursos com outras threads do mesmo processo. Esse compartilhamento, apesar de vantajoso para comunicação, também exige cuidados com sincronização e consistência.

Falando em sincronização, esse é o mecanismo que garante que várias threads acessando os mesmos recursos o façam de maneira controlada. Classes ou métodos são considerados thread-safe quando funcionam corretamente mesmo sendo acessados por várias threads simultaneamente. O Java nos oferece ferramentas como o "synchronized", locks como o ReentrantLock, classes atômicas como o AtomicInteger, e a keyword "volatile" para lidar com esses cenários.

A evolução da concorrência no Java é rica. Do suporte básico às threads nas primeiras versões até a chegada do pacote `java.util.concurrent` no Java 5, passando pelo Fork/Join Framework no Java 7, `CompletableFuture` e `Parallel Streams` no Java 8, e culminando com as Virtual Threads no Java 21. Essas inovações trouxeram mais performance, escalabilidade e uma API mais simples para lidar com tarefas concorrentes.

Hoje, o Java moderno oferece diferentes tipos de Thread Pools através do `ExecutorService`, como:

- Fixed Thread Pool: ideal para workloads previsíveis com recursos limitados.
- Cached Thread Pool: cria e reutiliza threads conforme a necessidade, sendo ideal para workloads imprevisíveis.
- Scheduled Thread Pool: executa tarefas agendadas em intervalos definidos.
- Single Thread Executor: para execução sequencial de tarefas.

Saber quando usar cada tipo é essencial para evitar desperdício de recursos e problemas de performance.

As collections thread-safe, como o `ConcurrentHashMap` e o `CopyOnWriteArrayList`, também são fundamentais em ambientes concorrentes. Elas permitem operações eficientes sem a necessidade de sincronização externa. O uso dessas estruturas é uma forma de aproveitar soluções já otimizadas por especialistas.

Temos ainda os synchronizers como `CountDownLatch`, `CyclicBarrier` e `Semaphore`, que permitem coordenar a execução entre threads. Esses

mecanismos são essenciais para cenários onde é necessário que threads esperem umas pelas outras ou limitem o acesso a recursos compartilhados.

O Fork/Join Framework é uma ferramenta poderosa para dividir tarefas grandes em subtarefas menores, aproveitando máximo paralelismo. Baseado no algoritmo WorkStealing, ele balanceia automaticamente as threads para otimizar a execução.

O CompletableFuture trouxe uma forma mais fluida de escrever código assíncrono, com encadeamento de tarefas, tratamento de exceções e composição de operações. Já o Parallel Stream permite paralelizar operações sobre coleções de maneira simples, tirando proveito do Fork/Join por baixo dos panos.

As Virtual Threads, uma das maiores inovações recentes, representam um novo modelo de execução onde as threads são gerenciadas diretamente pela JVM, e não mais pelo sistema operacional. Isso permite escalar para milhões de threads com consumo muito menor de recursos. A tendência é que no futuro, muito do que hoje fazemos com código assíncrono possa ser feito com código mais simples, usando virtual threads.

Boas práticas incluem o uso de objetos imutáveis, encapsulamento adequado do estado compartilhado, cópias defensivas, uso de operações atômicas e variáveis thread-local. Erros comuns como deadlocks e race conditions devem ser evitados com planejamento e sincronização adequada.

Outro ponto é a importância do dimensionamento correto dos pools, controle da granularidade das tasks e o uso de mecanismos como throttling para evitar sobrecarga do sistema. O monitoramento constante é essencial para identificar gargalos e otimizar o desempenho.

Na prática, concorrência é utilizada em servidores web, plataformas financeiras, big data, jogos, sistemas embarcados e muito mais. Tecnologias como CompletableFuture, Virtual Threads, Apache Kafka, Flink, entre outras, viabilizam esses cenários.

O futuro da concorrência no Java passa por uma adoção crescente das Virtual Threads, estabilização da Structured Concurrency, melhorias de performance na JVM e evolução das ferramentas de monitoração. O projeto Loom é o epicentro dessas transformações.

Esse foi nosso overview. A partir daqui, vamos aprofundar em cada um desses tópicos para que você esteja realmente preparado para aplicar concorrência de forma eficiente no seu dia a dia.

Capítulo 2: Executors E Thread Pools

Nesta aula, vamos aprofundar o conhecimento sobre executors e thread pools, dois conceitos fundamentais para o gerenciamento eficiente de threads na linguagem Java. Se no capítulo anterior fizemos um panorama geral sobre concorrência, agora é hora de explorar detalhadamente um dos pilares que tornam isso possível na prática.

A grande sacada do Executor Framework é a abstração que ele oferece sobre o gerenciamento de threads. Introduzido no Java 5, esse framework faz parte do pacote `java.util.concurrent` e resolve um problema clássico: o custo de criação e destruição de threads é alto. Ao utilizar executors, você separa a lógica da tarefa da lógica de execução e reaproveitamento de threads, ganhando eficiência e clareza no código.

A interface mais simples é a `Executor`, que possui apenas um método: `execute(Runnable command)`. Esse método é suficiente para permitir a submissão de tarefas sem a necessidade de se preocupar com a criação direta de threads. Porém, no dia a dia, o mais comum é trabalharmos com a interface `ExecutorService`, que estende `Executor` e traz métodos adicionais como `submit`, `shutdown`, `shutdownNow`, entre outros, permitindo um controle mais completo sobre o ciclo de vida das threads.

Outro componente essencial é a classe utilitária `Executors`, que fornece métodos factory para criação de instâncias de `ExecutorService`, com diferentes tipos de comportamento conforme o tipo de workload esperado. Aqui é onde entram os vários tipos de thread pools.

O `newFixedThreadPool` cria um pool com número fixo de threads. Ele é especialmente útil para workloads do tipo CPU-bound, onde o objetivo é manter todos os núcleos da CPU ocupados com mínima troca de contexto. No entanto, é preciso cuidado, pois ele usa uma `LinkedBlockingQueue` ilimitada internamente. Se muitas tarefas forem submetidas rapidamente, o risco de `OutOfMemoryError` é real.

Já o `newCachedThreadPool` é dinâmico: cria novas threads conforme necessário e reaproveita aquelas que estiverem ociosas por até 60 segundos. Ideal para workloads imprevisíveis e tarefas de curta duração. Mas também traz riscos: por não limitar a quantidade de threads, pode rapidamente esgotar recursos do sistema se mal utilizado.

O `newSingleThreadExecutor` garante que as tarefas sejam executadas em ordem e uma por vez. É uma opção válida quando se deseja controle estrito da ordem de execução ou se quer evitar qualquer tipo de concorrência na execução.

Temos também o `newScheduledThreadPool`, que adiciona agendamento de execuções. Ele permite disparar tarefas após um delay ou de forma periódica. Aqui, o método `scheduleAtFixedRate` tenta manter uma frequência constante, mesmo que a execução anterior não tenha terminado, o que pode ser perigoso. Já `scheduleWithFixedDelay` aguarda a conclusão da tarefa anterior antes de iniciar a próxima, sendo uma opção mais segura na maioria dos casos.

Embora seja tentador manipular diretamente os thread pools, isso deve ser evitado na maioria dos cenários de produção. Para situações mais complexas, recomenda-se utilizar a classe `ThreadPoolExecutor`, que oferece configurações detalhadas como tamanho mínimo e máximo do pool, tempo de vida de threads, entre outros parâmetros.

Agora, como dimensionar corretamente o tamanho de um thread pool? A resposta depende se suas tarefas são CPU-bound ou IO-bound.

Tarefas CPU-bound são aquelas que usam intensamente o processador, como cálculos, compressão de dados, processamento de imagem, etc. Nesses casos, mais threads do que núcleos causam degradação de performance por troca excessiva de contexto. A boa prática é configurar o thread pool com o mesmo número de núcleos ou um pouco menos.

Tarefas IO-bound envolvem operações que bloqueiam temporariamente a thread, como chamadas de rede, acesso a disco, banco de dados, etc. Aqui, a CPU fica ociosa enquanto aguarda o retorno da operação. Logo, é indicado ter

mais threads que núcleos para aproveitar melhor os recursos enquanto algumas estão em espera.

Brian Goetz, arquiteto da plataforma Java, sugere uma fórmula aproximada para dimensionamento de pools: $\text{Número de CPUs} * (1 + \text{Tempo de Espera} / \text{Tempo de Processamento})$. Essa equação ajuda a calcular o tamanho ideal do pool com base na relação entre tempo em que a thread fica ociosa e o tempo que realmente trabalha.

Esse tipo de reflexão sobre alocação de threads mostra bem o quanto as Platform Threads (threads baseadas no sistema operacional) são limitadas. Cada thread tradicional consome recursos do sistema, mesmo quando bloqueada. É um modelo caro. Essa limitação foi uma das principais motivações para o surgimento das Virtual Threads, que vamos explorar com mais profundidade em um capítulo futuro.

De forma resumida, as Virtual Threads são gerenciadas pela JVM, não mais pelo sistema operacional. Elas são montadas e desmontadas dinamicamente, podendo ser executadas por uma thread portadora (carrier thread). Quando ocorre uma operação bloqueante, a JVM remove a virtual thread da carrier, armazena seu estado na heap, e libera a carrier para outra execução. Isso permite escalar para milhões de threads com consumo muito menor de recursos.

Um ponto que era crítico até o Java 24 era o pinning: o risco de uma virtual thread prender a carrier thread ao usar blocos `synchronized`. Esse problema foi resolvido no Java 24, tornando o uso das virtual threads ainda mais robusto.

Por fim, é essencial entender como fazer o desligamento correto dos executors. Mesmo quando não estão em uso, os thread pools continuam consumindo recursos se não forem encerrados adequadamente. Para isso, temos:

- `shutdown()`: para um encerramento gracioso. Ele para de aceitar novas tarefas, mas permite que as existentes sejam concluídas.
- `shutdownNow()`: tentativa agressiva de encerramento. Tenta interromper as tarefas em execução e retorna as que ainda não começaram.

- `awaitTermination(timeout, unit)`: permite que a thread principal aguarde por um tempo para que as tarefas se encerrem.

Executors e thread pools são ferramentas poderosas, mas devem ser usadas com consciência. Entender como dimensionar, quando usar cada tipo e como finalizar corretamente são passos fundamentais para o sucesso de aplicações concorrentes robustas e eficientes.

No próximo capítulo, vamos aprofundar ainda mais em um dos temas que mais causam dúvidas entre desenvolvedores: o uso correto de sincronização e os riscos relacionados a isso.

Capítulo 3: Concurrent Collections

Neste capítulo, vamos explorar o uso de coleções concorrentes em Java, um recurso essencial para quem precisa trabalhar com estruturas de dados em ambientes de alta concorrência. Ao contrário das coleções tradicionais como

`ArrayList` ou `HashMap`, que não foram projetadas para operações simultâneas de múltiplas threads, as coleções concorrentes surgem como soluções otimizadas e seguras para esse tipo de cenário.

O primeiro ponto que precisamos entender é o desafio do estado compartilhado. Toda coleção é, na prática, um conjunto de estados acessíveis por diferentes pontos do sistema. Quando esses acessos passam a ocorrer de forma concorrente, os problemas começam a surgir: inconsistências, corrupção de dados e exceções como a temida `ConcurrentModificationException` são apenas alguns exemplos.

Historicamente, as primeiras tentativas de tornar coleções thread-safe no Java vieram com `Collections.synchronizedList` e `Collections.synchronizedMap`. Essas abordagens funcionam ao envolver os métodos das coleções em blocos `synchronized`, criando um lock global. O problema é que esse tipo de lock tem uma granularidade muito grossa, pois toda a coleção é bloqueada mesmo para operações simples. Além disso, ainda que garantam alguma segurança, essas abordagens não evitam exceções de modificação concorrente durante iterações, exigindo sincronização manual adicional.

A solução mais eficaz está nas coleções concorrentes propriamente ditas, projetadas especificamente para esse tipo de ambiente. Vamos ver as principais delas.

ConcurrentHashMap

O `ConcurrentHashMap` é uma das estruturas mais utilizadas quando falamos de coleções otimizadas para concorrência. Sua vantagem em relação ao `synchronizedMap` vem da forma como ele gerencia os locks. Em vez de bloquear toda a coleção, ele aplica uma estratégia de locking de granularidade fina.

Antes do Java 8, isso era feito através do "lock stripping": o mapa era dividido em segmentos, cada um com seu próprio lock. A partir do Java 8, o controle passou a ser ainda mais refinado, com locks aplicados diretamente no nó inicial de cada bucket. As operações de escrita são controladas por locks localizados, enquanto as de leitura são não bloqueantes, com suporte ao uso da palavra-

chave `volatile`, garantindo visibilidade entre threads. Essa arquitetura permite alta taxa de throughput mesmo em sistemas com alto nível de concorrência.

CopyOnWriteArrayList

Quando a necessidade é realizar muitas leituras e poucas escritas, o `CopyOnWriteArrayList` é uma excelente opção. A ideia aqui é simples e engenhosa: ao invés de bloquear a estrutura para proteger acessos concorrentes, ela trabalha com uma cópia imutável do array interno.

Durante as leituras, não há sincronização alguma. A thread simplesmente acessa o snapshot atual do array, declarado como `volatile`, o que garante visibilidade segura. Quando uma escrita é necessária, a estrutura segue os seguintes passos: adquire um lock, faz uma cópia completa do array, aplica a modificação na cópia, atualiza a referência para essa nova cópia de forma atômica, e por fim libera o lock.

O grande benefício disso é que iterações sempre acontecem sobre uma cópia imutável do array original. Isso elimina o risco de `ConcurrentModificationException`, pois o iterador nunca é afetado por mudanças que ocorram posteriormente em outras threads.

BlockingQueue

No modelo produtor-consumidor, a `BlockingQueue` é a estrutura ideal. Essa interface resolve com elegância o problema da coordenação entre threads que produzem e threads que consomem tarefas. Diferente de filas comuns, a `BlockingQueue` oferece métodos que bloqueiam as threads de forma controlada até que possam prosseguir com a operação desejada.

Quando um produtor chama o método `put` para adicionar um item e a fila está cheia, ele é bloqueado até que um consumidor libere espaço. Do outro lado, se um consumidor tenta fazer um `take` e a fila está vazia, ele também é bloqueado até que haja um item disponível. Essa abordagem elimina a necessidade de usar `wait`, `notify` ou sincronização manual. A fila cuida de tudo para você.

Esse comportamento não apenas simplifica o código, mas também garante maior estabilidade e previsibilidade em ambientes concorrentes, onde o desacoplamento entre produção e consumo é desejável.

Conclusão

Coleções concorrentes são ferramentas indispensáveis quando estamos lidando com multithreading em Java. Elas resolvem problemas reais de concorrência, mantendo a integridade dos dados e a performance das aplicações. Mais do que apenas garantir que duas threads não se atrapalhem, essas coleções foram pensadas para oferecer desempenho em ambientes com alto volume de acesso simultâneo.

Entender quando usar cada uma delas é essencial. Se você precisa de acesso concorrente a mapas, `ConcurrentHashMap` é sua escolha. Para listas onde a leitura é muito mais frequente que a escrita, `CopyOnWriteArrayList` vai oferecer melhor resultado. E em padrões de produtor-consumidor, `BlockingQueue` é a solução natural.

No próximo capítulo, vamos nos aprofundar em mecanismos de sincronização, incluindo locks, semáforos, barriers e outros recursos críticos para a coordenação eficaz entre threads.

Capítulo 4: Synchronizers

Neste capítulo, vamos explorar os synchronizers, um conjunto de ferramentas fundamentais quando o assunto é coordenação entre threads em ambientes concorrentes. Enquanto blocos `synchronized` e locks tradicionais cumprem bem o papel de garantir exclusão mútua e proteger estados compartilhados, muitas situações exigem formas de coordenação mais elaboradas. É nesse contexto que entram os synchronizers, oferecendo abstrações de mais alto nível, com a robustez esperada das APIs do Java.

O primeiro synchronizer que costumo apresentar é o `CountDownLatch`. Sua operação é simples, mas extremamente poderosa. Funciona como uma contagem regressiva que bloqueia uma ou mais threads até que um determinado número de eventos tenha ocorrido. Por exemplo, podemos inicializar um `CountDownLatch` com o valor 10. Imagina que temos 10 threads trabalhando em tarefas diferentes. Enquanto isso, outras 5 threads dependem da conclusão dessas 10 para continuar seu processamento. As threads dependentes chamam o método `await` e ficam bloqueadas, aguardando. Cada uma das 10 threads, ao finalizar sua tarefa, chama o método `countDown`, reduzindo o contador em um. Quando esse contador chega a zero, todas as threads bloqueadas são liberadas simultaneamente. Um caso clássico de uso seria a inicialização de serviços em uma aplicação: a thread principal aguarda que todos os serviços estejam prontos antes de seguir. É como um readiness manual, como ocorre em sistemas como o Kubernetes.

Outro synchronizer bastante interessante é o `CyclicBarrier`. Diferente do `CountDownLatch`, aqui não temos threads esperando eventos externos, mas sim threads esperando umas pelas outras. É como se todas combinassem um ponto de encontro: nenhuma pode prosseguir antes que todas estejam presentes. Inicializamos o `CyclicBarrier` com o número de threads esperadas, que são chamadas de "parties". Cada thread chama `await`, ficando bloqueada. Quando a última thread também chama `await`, a barreira é rompida e todas são liberadas para continuar. É uma ferramenta muito útil quando temos computações paralelas que precisam sincronizar em etapas específicas. Um exemplo clássico é a coleta de dados de diferentes fontes: só podemos continuar quando todas as fontes tiverem respondido.

O `Semaphore` é mais uma opção crucial no arsenal de synchronizers. Ele é usado para limitar o número de threads que podem acessar simultaneamente um recurso limitado. Imagine um pool de conexões com um banco de dados que suporta até 100 conexões simultâneas. Se 10.000 usuários tentarem acessar ao mesmo tempo, precisamos de um mecanismo para controlar isso. Com o `Semaphore`, podemos criar um controle de acesso com 100 permissões. Cada

thread chama `acquire` para solicitar uma permissão. Se houver permissão disponível, ela prossegue. Caso contrário, a thread é bloqueada até que uma permissão seja liberada por outra thread que chame `release`. Esse tipo de controle é ideal para qualquer recurso escasso, como conexões de rede ou arquivos abertos. Apesar de não estar diretamente relacionado ao pinning de Virtual Threads, é um excelente aliado para lidar com limitações de acesso, inclusive nesse novo modelo de execução.

Por fim, temos o `Phaser`, o synchronizer mais moderno entre os apresentados aqui. Ele combina as funcionalidades do `CountDownLatch` e do `CyclicBarrier`, oferecendo maior flexibilidade. Introduzido no Java 7, o `Phaser` permite a coordenação entre threads de maneira mais dinâmica. Enquanto os outros synchronizers exigem definir previamente o número de participantes, o `Phaser` permite registrar e desregistrar threads durante a execução. Isso é feito por meio dos métodos `register` e `arriveAndDeregister`. Uma thread pode se juntar à coordenação a qualquer momento, e sair dela quando não for mais necessária. Isso é extremamente útil em algoritmos paralelos complexos, onde a quantidade de participantes pode variar entre fases de processamento. Como o próprio nome sugere, o `Phaser` organiza a execução em fases, permitindo maior controle sobre as etapas de coordenação.

De forma geral, os synchronizers oferecem soluções precisas para desafios comuns em ambientes multithread. Saber quando e como usar cada um deles pode ser a diferença entre um sistema concorrente robusto e um repleto de bugs sutis. O `CountDownLatch` é ideal para inicialização sequencial. O `CyclicBarrier` é perfeito para sincronização em etapas. O `Semaphore` é excelente para controle de acesso a recursos escassos. E o `Phaser` brilha em cenários dinâmicos e iterativos.

No próximo capítulo, vamos entrar em um tema que desperta muito interesse e também confusão entre desenvolvedores: o modelo Fork/Join, sua estratégia de paralelismo e a arquitetura baseada em work stealing.

Capítulo 5: Fork/Join Framework

Neste capítulo, vamos explorar o Fork/Join Framework, uma das ferramentas mais especializadas do Java para execução paralela de tarefas. Introduzido no Java 7, esse framework foi projetado para resolver problemas de alta complexidade, nos quais é vantajoso dividir uma grande tarefa em diversas subtarefas independentes. Essa abordagem segue o padrão "divide and conquer" (dividir para conquistar), permitindo uma resolução eficiente e paralela das partes do problema.

A ideia central do Fork/Join é bastante intuitiva: pegar um problema grande, quebrá-lo em partes menores (fase "fork") e depois combinar os resultados dessas partes (fase "join"). Essa estratégia é particularmente eficiente em operações como ordenação de grandes arrays, transformações de dados em massa, algoritmos de busca em árvore e outros cenários de computação pesada.

O Fork/Join Framework é uma especialização da interface `ExecutorService`, e sua diferença está no algoritmo de balanceamento de carga que implementa: o chamado Work Stealing. Esse algoritmo permite que threads ociosas "roubem" tarefas de outras threads mais ocupadas, mantendo um equilíbrio dinâmico na execução paralela.

ForkJoinPool e o funcionamento interno

No coração do framework está o `ForkJoinPool`, que é o responsável por gerenciar o conjunto de worker threads. Em geral, temos uma thread por CPU disponível, e cada thread possui sua própria fila de tarefas, implementada como uma estrutura `Deque` (Double Ended Queue). Isso permite a operação em duas pontas: a cabeça e a cauda da fila.

Cada worker thread consome suas próprias tarefas no modo LIFO (Last In, First Out), ou seja, ela pega para executar a tarefa que entrou por último na sua própria fila. Essa escolha favorece o uso de cache, pois os dados recém-alocados tendem a estar mais "quentes" na CPU.

Quando uma thread termina suas tarefas e sua fila fica vazia, ela não fica ociosa: ela se torna uma thread ladra. Nesse modo, ela busca tarefas em outras filas, mas sempre pela cauda dessas filas, utilizando o modo FIFO (First In, First Out). Isso reduz a contenção, pois a thread proprietária consome pela cabeça e as ladras pela cauda. Além disso, tarefas mais antigas tendem a estar menos otimizadas em cache, então é eficiente que sejam alocadas para outras threads.

RecursiveTask e RecursiveAction

Dentro do Fork/Join Framework, usamos classes especializadas para representar as tarefas. As duas principais são:

- `RecursiveTask<V>`: usada quando a tarefa precisa retornar um valor.

- **RecursiveAction**: usada quando a tarefa não retorna nada, apenas realiza alguma ação.

Ambas exigem a implementação do método **compute**, onde se define a lógica da tarefa. É nesse ponto que aplicamos efetivamente o "divide and conquer". A lógica básica é verificar se o problema é pequeno o suficiente para ser resolvido diretamente; caso contrário, ele é dividido em subtarefas menores.

Essa decisão sobre o que é considerado "pequeno o suficiente" é fundamental. Um corte mal planejado pode desperdiçar recursos, seja por não paralelizar o suficiente ou por gerar overhead com muitas subtarefas.

Execução de tarefas e padrão de invocação

Na prática, a execução de tarefas com Fork/Join começa com a criação de uma instância do **ForkJoinPool**. O construtor padrão utiliza o número de processadores disponíveis, que pode ser obtido com **Runtime.getRuntime().availableProcessors()**.

A tarefa raiz, que é a entrada principal da lógica recursiva, é enviada ao pool por meio do método **invoke**. Essa chamada é síncrona: bloqueia a thread chamadora até que todas as subtarefas sejam concluídas e o resultado final esteja disponível.

Durante a implementação do **compute**, é comum adotar o seguinte padrão: dividir o problema em duas partes, invocar uma das subtarefas usando **fork()** (possibilitando que ela seja roubada por outra thread) e processar a outra localmente usando **compute()**. Ao final, é feita uma chamada a **join()** para aguardar o resultado da tarefa delegada.

Essa estratégia reduz a sobrecarga e melhora a performance, pois evita o custo de gerenciamento desnecessário de muitas threads para tarefas simples.

ForkJoinPool comum e seus riscos

A partir do Java 8, o Java passou a disponibilizar um pool compartilhado chamado **ForkJoinPool.commonPool()**. Esse pool é estático e compartilhado

entre toda a JVM, sendo usado por padrão por APIs como `Parallel Streams` e `CompletableFuture`.

Apesar de ser conveniente, o uso do `commonPool` exige cuidado. Ele é otimizado para tarefas curtas e CPU-bound. Caso você envie para esse pool tarefas IO-blocking ou muito demoradas, pode acabar bloqueando suas threads, comprometendo outras partes da aplicação que dependem do mesmo pool. Isso pode resultar em deadlocks ou degradação grave de performance, sendo um problema muitas vezes difícil de diagnosticar.

Por isso, para workloads críticos, longos ou bloqueantes, a melhor prática é criar um `ForkJoinPool` dedicado. Isso garante isolamento, resiliência e protege a performance geral do sistema.

Considerações finais

O Fork/Join Framework é uma ferramenta poderosa para paralelismo em Java. Seu modelo de execução baseado em divide and conquer, aliado ao algoritmo de work stealing, garante alta eficiência em distribuição de carga.

Saber diferenciar entre `RecursiveTask` e `RecursiveAction`, escolher corretamente os pontos de corte da recursão e entender o funcionamento do pool são elementos chave para o uso eficaz dessa ferramenta.

No próximo capítulo, vamos explorar o funcionamento das Virtual Threads, suas diferenças em relação às platform threads e como elas estão revolucionando o modelo de concorrência na plataforma Java.

Capítulo 6: CompletableFuture

Neste capítulo, vamos explorar uma das ferramentas mais conhecidas e poderosas para o trabalho com concorrência no Java moderno: o `CompletableFuture`. Introduzido no Java 8 como uma evolução da interface `Future` (disponível desde o Java 5), o `CompletableFuture` representa um grande salto em termos de flexibilidade, legibilidade e composição de tarefas assíncronas.

Enquanto o `Future` original era limitado e bloqueante – dependendo do uso do método `get()` que travava a thread até o resultado estar disponível –, o `CompletableFuture` se apoia em uma nova interface chamada `CompletionStage`. Essa interface permite uma programação declarativa e reativa, com suporte a mais de 50 métodos que facilitam a criação de pipelines assíncronos robustos e não bloqueantes.

A criação de um `CompletableFuture` é feita por meio de métodos factory estáticos. Os dois principais são `runAsync`, usado para tarefas que executam ações sem retorno de resultado, e `supplyAsync`, usado quando a tarefa produz

um valor ao final. Ambos, por padrão, executam na `ForkJoinPool.commonPool()`, mas é possível passar um executor personalizado, o que é altamente recomendado para tarefas IO-blocking. Nesse contexto, pode-se usar um `FixedThreadPool` ou um executor baseado em Virtual Threads, como o `Executors.newVirtualThreadPerTaskExecutor()`.

A verdadeira força do `CompletableFuture` está no seu modelo de encadeamento. Com métodos como `thenApply`, `thenAccept`, `thenRun` e suas variantes `async`, é possível construir pipelines de transformação e execução em estágios distintos. O `thenApply` transforma o resultado de uma operação, o `thenAccept` consome esse resultado sem retornar nada, e o `thenRun` executa uma ação independente, sem depender do valor anterior.

A diferença entre as versões normais e `async` está na thread onde a operação subsequente ocorre. As versões normais utilizam a mesma thread que completou o estágio anterior, enquanto as versões `async` submetem a nova tarefa a um pool de threads. Esse controle é crucial para evitar travamentos ou uso excessivo da thread atual em operações demoradas.

Uma distinção importante no uso do `CompletableFuture` é entre `thenApply` e `thenCompose`. O primeiro é utilizado quando a função de transformação retorna um valor direto. O segundo, por sua vez, é essencial quando a transformação também é assíncrona, retornando outro `CompletableFuture`. O `thenCompose` evita a criação de uma estrutura aninhada de `CompletableFuture<CompletableFuture<T>>`, realizando um *flatten* automático.

Outro aspecto avançado é a composição de múltiplos futures. Para isso, temos o `thenCombine`, ideal quando temos duas operações independentes que devem ser unificadas após concluídas. Além disso, os métodos `allOf` e `anyOf` oferecem um mecanismo de sincronização para diversos futures:

- `allOf` aguarda a conclusão de todos os futures fornecidos. É útil para cenários de agregação, onde vários resultados precisam ser consolidados antes de continuar.
- `anyOf` libera o fluxo assim que o primeiro future é completado, ideal para competição entre tarefas, como obter a primeira resposta entre várias APIs de terceiros.

Um exemplo comum de uso do `allOf` é integrar com a API de `Streams`, mapeando um conjunto de `CompletableFutures`, aplicando `join()` para aguardar os resultados e consolidando tudo em uma lista.

Por fim, o tratamento de erros no `CompletableFuture` é um de seus recursos mais interessantes. Ele permite um controle fluido e funcional de exceções, substituindo a necessidade de blocos `try-catch` verbosos. Os dois principais mecanismos são:

- `exceptionally`: intercepta uma exceção e retorna um valor de fallback. Caso nenhuma exceção ocorra, ele é ignorado.
- `handle`: executado sempre, com ou sem erro, recebendo dois parâmetros: o resultado (ou `null` em caso de falha) e a exceção (ou `null` em caso de sucesso). É semelhante a um bloco `finally`, mas com mais controle.

O `CompletableFuture` representa, portanto, uma mudança significativa no modelo de programação concorrente em Java, trazendo legibilidade, eficiência e controle para operações assíncronas. Seu uso adequado pode transformar a forma como construímos pipelines de execução, garantindo não apenas performance, mas também uma experiência de desenvolvimento mais fluida e moderna.

No próximo capítulo, vamos abordar as Virtual Threads, discutindo como elas estão mudando o paradigma de concorrência e o impacto que trazem para aplicações modernas na plataforma Java.

Capítulo 7: Parallel Streams

Neste capítulo, abordo o uso de Parallel Streams, um recurso introduzido no Java 8 que permite paralelizar operações sobre coleções de forma extremamente simples. Com apenas uma linha de código, é possível transformar um pipeline sequencial em paralelo, utilizando os métodos `.parallel()` ou `.parallelStream()`. Essa simplicidade é atraente, mas como ocorre frequentemente, por trás dessa facilidade existem diversas armadilhas e considerações importantes que precisam ser compreendidas para evitar problemas de desempenho ou mesmo bugs graves.

A base de funcionamento do Parallel Streams é o Fork/Join Framework, já discutido em um capítulo anterior. Quando invocamos um stream paralelo, o Java cria um `Splitterator`, uma interface especializada para divisão eficiente de dados. Essa divisão funciona muito bem com estruturas contíguas e indexáveis, como `ArrayList` ou arrays primitivos, pois essas estruturas podem ser particionadas com cálculos simples de índice. Por outro lado, estruturas baseadas em nós, como `LinkedList`, são extremamente ineficientes nesse contexto, pois exigem percorrer toda a estrutura até encontrar o ponto de divisão.

Uma vez que os dados são divididos, cada fragmento é processado como uma tarefa dentro do `ForkJoinPool.commonPool()`, o mesmo pool compartilhado por `CompletableFuture` e outras APIs da JVM. Esse compartilhamento exige cautela, pois um uso indevido pode levar à degradação de performance ou até mesmo à paralisação da JVM. Como regra, não se deve utilizar Parallel Streams para

tarefas que envolvam I/O bloqueante, pois threads do pool que ficam bloqueadas impedem a execução de outras tarefas, podendo saturar completamente o pool.

Então, quando é que é adequado utilizar Parallel Streams? Primeiro, quando temos tarefas puramente CPU-bound, ou seja, operações que exigem apenas processamento e não acessam recursos externos. Segundo, quando lidamos com grandes volumes de dados e cada elemento exige um processamento considerável. Nesse contexto, existe um modelo chamado "NQ" que pode ajudar na decisão: se o número de elementos (N) multiplicado pelo trabalho por elemento (Q) for superior a 10.000, é provável que o uso de paralelismo compense.

Outro critério importante é a capacidade de divisão eficiente da fonte de dados. Como mencionado, usar estruturas como `ArrayList` é altamente recomendado. Além disso, é fundamental que as operações do stream sejam stateless, ou seja, não alterem estados externos. Streams paralelos não garantem ordem de execução, e acessos concorrentes a estados mutáveis podem causar race conditions, corrupção de dados ou exceções como `ArrayIndexOutOfBoundsException`.

Para agregar resultados em streams paralelos, o correto é utilizar operações como `.collect()` ou `.reduce()`, que foram projetadas para lidar com concorrência de forma segura. Evite a utilização de lambdas que modificam coleções externas ou variáveis que não são thread-safe. Esse tipo de prática, embora funcione em streams sequenciais, pode causar comportamentos imprevisíveis e difíceis de depurar em streams paralelos.

Além das boas práticas, é essencial que toda adoção de Parallel Streams seja precedida por medições reais. Benchmarking é a única forma confiável de saber se a paralelização está de fato trazendo ganhos. Não é incomum que uma stream paralela seja mais lenta que sua versão sequencial, especialmente se as condições ideais não forem atendidas. Portanto, tenha um baseline de performance antes e depois da paralelização para avaliar sua eficácia.

Em resumo, o Parallel Streams é uma ferramenta poderosa e elegante para explorar o paralelismo, mas deve ser usada com responsabilidade. Evite I/O bloqueante, garanta que as operações sejam stateless, use estruturas de dados

apropriadas e faça benchmarking. Não é porque é simples que deve ser usado indiscriminadamente. Saber quando **não** usar é tão importante quanto saber como usar.

No próximo capítulo, vamos discutir um dos avanços mais relevantes no modelo de concorrência do Java moderno: as Virtual Threads. Elas prometem mudar radicalmente a forma como lidamos com concorrência, oferecendo um modelo leve, escalável e muito mais simples de gerenciar.

Capítulo 8: Virtual Threads

Neste capítulo, apresento um dos avanços mais significativos da plataforma Java nas últimas décadas: as Virtual Threads. Essa funcionalidade, resultado do Project Loom, promete transformar radicalmente a forma como lidamos com concorrência em Java, aproximando desempenho de escalabilidade com simplicidade de código.

Para começar, gosto de destacar uma frase do Brian Goetz, uma das principais referências na linguagem Java: “Para usar Virtual Threads efetivamente, há mais a ser desaprendido do que aprendido”. Isso porque, até o surgimento das Virtual Threads, nosso modelo mental de concorrência era totalmente ancorado nas Platform Threads. Essas threads clássicas, representadas pela API `java.lang.Thread`, são wrappers diretos sobre as threads do sistema operacional. Isso significa que elas herdam todas as limitações do SO, incluindo alto consumo de memória, troca de contexto custosa e subutilização do hardware em cenários de grande concorrência.

O problema principal com Platform Threads é que, no modelo thread-per-request, cada requisição ocupa uma thread durante todo seu ciclo de vida. Isso gera limitações claras de throughput, pois a concorrência fica diretamente atrelada ao número de threads disponíveis. Conforme estabelece a Lei de Little, a concorrência é igual ao throughput multiplicado pela duração da requisição. Logo, para aumentar o throughput, era necessário aumentar o número de threads, o que implicava aumento de custo de hardware.

Como alternativa, adotou-se largamente a programação assíncrona. Ao permitir que uma thread fosse liberada durante operações de I/O, como acesso a disco ou redes, conseguimos atender mais requisições com menos threads. Porém, esse modelo traz desafios significativos: código menos intuitivo, necessidade de

novas APIs e, principalmente, perda de compatibilidade com ferramentas tradicionais do ecossistema Java, como debuggers, profilers e stack traces.

E é nesse contexto que surgem as Virtual Threads, propostas como uma alternativa que combina o melhor dos dois mundos: código simples, legível e observável com alta escalabilidade e aproveitamento de hardware. Disponíveis em sua versão final a partir do Java 21 (JEP 444), as Virtual Threads são implementadas integralmente dentro da JVM e não são wrappers sobre threads do sistema operacional. Isso significa que a JVM pode gerenciá-las de forma mais eficiente, sem as limitações herdadas do SO.

Uma única thread de sistema operacional, chamada de carrier thread, pode carregar milhares ou até milhões de Virtual Threads. Isso permite uma utilização quase ótima do hardware, eliminando o antigo problema de subutilização. Mais importante ainda, a API continua a mesma: você pode usar a mesma sintaxe de `Thread`, `Runnable`, `Callable` e `ExecutorService`, com alguns novos métodos adicionados, como `Thread.startVirtualThread()` ou `Executors.newVirtualThreadPerTaskExecutor()`.

Com Virtual Threads, voltamos a um modelo de programação tradicional, bloqueante, com todo o suporte de ferramentas do Java, mas com escalabilidade comparável (ou superior) à do modelo assíncrono. Mas isso não significa que não haja cuidados a tomar. O uso efetivo das Virtual Threads requer uma mudança de mentalidade, especialmente para quem trabalhou extensivamente com Thread Pools e otimizações de concorrência tradicionais.

Para adotar Virtual Threads corretamente, compartilho quatro recomendações fundamentais:

1. **Utilize APIs bloqueantes com sabedoria:** diferentemente do modelo reativo, aqui o uso de APIs bloqueantes é aceitável e, em muitos casos, preferível. Isso preserva a simplicidade do código e a integração com ferramentas. As Virtual Threads toleram bloqueios, pois são leves e fáceis de reatribuir dentro da JVM.
2. **Evite Thread Pools convencionais:** no modelo tradicional, pools são usados para limitar o número de threads. Com Virtual Threads, o ideal é usar o padrão thread-per-task, em que cada tarefa é executada em sua

própria virtual thread, sem competição por threads limitadas. Isso promove maior paralelismo e melhor utilização dos recursos.

3. **Fuja do pinning:** pinning ocorre quando uma Virtual Thread fica presa a uma carrier thread, como em blocos `synchronized` longos ou operações CPU-bound. Isso prejudica a escalabilidade. A partir do Java 24 (JEP 491), o problema foi resolvido, mas é importante estar atento em versões anteriores.
4. **Evite monopolizar carrier threads:** não faça com que uma Virtual Thread bloqueie uma carrier thread por tempo excessivo, especialmente em tarefas intensivas em CPU ou I/O de longa duração. Isso pode levar a degradação de performance e esgotamento de recursos.

Outro ponto importante é a possibilidade de controlar concorrência com `Semaphore`, como já vimos em Synchronizers. Isso é essencial quando precisamos limitar o número de acessos simultâneos a recursos escassos, como conexões com banco de dados.

No horizonte próximo, temos o Structured Concurrency, que está em fase de preview (Java 24 e Java 25) e visa agrupar tarefas concorrentes em estruturas mais fáceis de gerenciar. Com ele, será possível controlar cancelamentos, exceções e execuções de forma coesa entre tarefas independentes. Esse recurso deve complementar ainda mais o modelo de Virtual Threads, oferecendo previsibilidade e melhor observabilidade.

Para quem usa o Quarkus, a adoção das Virtual Threads foi facilitada por meio da anotação `@RunOnVirtualThread`, que pode ser aplicada tanto em classes quanto em métodos. Isso permite ao runtime tentar executar automaticamente o código em Virtual Threads, revertendo para Platform Threads caso não seja possível.

Encerrando, as Virtual Threads representam uma mudança de paradigma na plataforma Java, permitindo que escrevamos código mais simples, mais rápido e mais eficiente, com compatibilidade total com o ecossistema existente. Elas não substituem a programação reativa, mas oferecem uma alternativa poderosa e escalável para novos projetos. O futuro da concorrência em Java passa, sem dúvida, pelas Virtual Threads.

Capítulo 9: Mão na Massa

Ao longo das aulas anteriores, eu optei por apresentar os conceitos teóricos de concorrência e multithreading em Java de forma isolada, sem me aprofundar em exemplos de código. Essa decisão foi proposital. Meu objetivo era garantir que todos os conceitos estivessem bem sedimentados antes de partirmos para a prática. Agora, com esse conhecimento consolidado, é hora de colocar a mão na massa. Esta aula é justamente o ponto de convergência entre tudo o que discutimos até aqui e a implementação prática.

Para facilitar seu aprendizado, preparei um material complementar contendo todos os exemplos explicados aqui. Cada trecho de código está comentado e separado por aula e por exemplo, seguindo a mesma estrutura que usamos ao longo do curso. Isso permite que você acompanhe e teste os códigos de forma organizada. Recomendo que, durante o estudo, abra esse material de apoio e execute os exemplos para entender o comportamento detalhado das soluções propostas.

Começando pela aula 2, tratamos de "Executors" e gerenciamento de threads. O primeiro exemplo apresentado é uma simulação de tarefas do tipo CPU bound, ou seja, que consomem intensivamente recursos da CPU. Neste caso, utilizamos um thread pool fixo, o `FixedThreadPool`, configurado com base na quantidade de núcleos da sua máquina. O código simula a execução de múltiplas tarefas intensas (como somas até um bilhão) de forma paralela. Essa configuração garante que o número de threads seja adequado ao ambiente, promovendo uma melhor utilização dos recursos disponíveis.

No segundo exemplo, demonstramos o uso de virtual threads com o `ExecutorService`. Aqui, ao invés de gerenciar explicitamente o pool de threads, utilizamos o `newVirtualThreadPerTaskExecutor`, que delega esse gerenciamento à JVM. Criamos 100.000 tarefas simples com um delay de um

segundo cada, para ilustrar a leveza e escalabilidade das virtual threads, que permitem um volume massivo de tarefas concorrentes com mínimo overhead.

O terceiro exemplo trata das threads agendadas, utilizando o `ScheduledThreadPoolExecutor`. Apresentamos três variações: execução com atraso fixo, execução com intervalo fixo a partir do início e execução com atraso fixo após a finalização da tarefa anterior. Cada uma dessas abordagens é útil para diferentes tipos de agendamentos em aplicações reais, como rotinas de verificação, sincronizações periódicas e manutenção de estado.

Por fim, no quarto exemplo dessa aula, abordamos o ciclo de vida do `ExecutorService`, comparando o uso pós-Java 19, com suporte ao `AutoCloseable`, ao uso tradicional em versões anteriores. Exploramos a importância do shutdown controlado para evitar vazamentos de recursos e garantir que as tarefas sejam finalizadas corretamente. Apresentamos uma abordagem usando `try-with-resources`, e outra manual, com controle refinado via `shutdown`, `awaitTermination` e `shutdownNow`.

Na aula 3, exploramos coleções concorrentes. Começamos comparando o uso do `SynchronizedMap` com o `ConcurrentHashMap`, evidenciando as vantagens de performance do segundo em ambientes com múltiplas threads. Utilizamos uma carga elevada de inserções para demonstrar o ganho significativo de desempenho do `ConcurrentHashMap`.

Em seguida, analisamos o comportamento do `CopyOnWriteArrayList`, ideal para cenários com muitas leituras e poucas alterações. Mostrei como ele gerencia a leitura de forma segura, criando cópias temporárias para evitar conflitos de concorrência. Apresentamos um exemplo onde notificações são enviadas enquanto novos listeners são adicionados, demonstrando a robustez da estrutura.

Também introduzimos o padrão `Producer-Consumer`, utilizando `BlockingQueue` em conjunto com virtual threads. Este padrão permite o desacoplamento entre produção e consumo de dados, promovendo maior escalabilidade. Simulamos um produtor que insere elementos na fila e um consumidor que os processa, demonstrando o comportamento de bloqueio quando a fila está cheia ou vazia.

A aula 4 nos levou ao estudo dos Synchronizers. Começamos com o `CountDownLatch`, que permite a sincronização de múltiplas threads aguardando até que um conjunto de eventos seja concluído. Mostrei como utilizar esse mecanismo para garantir que certos serviços estejam prontos antes de prosseguir com a execução.

Depois, abordamos o `CyclicBarrier`, onde múltiplas threads precisam atingir um ponto comum de execução antes de prosseguir. Mostrei como usá-lo em múltiplas fases, simulando ciclos onde as threads precisam sincronizar-se periodicamente.

Na sequência, exploramos o uso do `Semaphore`, um mecanismo clássico de controle de acesso a recursos limitados. Demonstrei como gerenciar múltiplas threads competindo por conexões de banco, permitindo que apenas três delas utilizassem o recurso simultaneamente.

Finalizamos com o `Phaser`, um dos synchronizers mais flexíveis. Ele combina funcionalidades do `CountDownLatch` e do `CyclicBarrier`, permitindo o registro e desregistro dinâmico de participantes. Mostrei como utilizá-lo em múltiplas fases, controlando a evolução de tarefas que nem sempre iniciam ou terminam ao mesmo tempo.

A aula 5 focou no Fork/Join Framework, com destaque para o algoritmo `Divide and Conquer`. Mostrei como dividir uma tarefa pesada – como somar elementos de um array com milhões de números – em subtarefas, que são executadas paralelamente e depois combinadas. Comparando o uso do `Common Pool` com pools customizados, expliquei os prós e contras de cada abordagem, incentivando o uso de pools isolados em cargas pesadas para evitar interferência em outros componentes da aplicação.

Na aula 6, mergulhamos no `CompletableFuture`. Apresentei os operadores `thenApply` e `thenCompose`, mostrando como evitar aninhamentos complexos e melhorar a legibilidade. Em seguida, demonstramos o `thenCombine` para combinar múltiplos resultados assíncronos, e o `allOf` para aguardar a finalização de várias tarefas em paralelo. Finalizamos com o tratamento de exceções assíncronas usando `exceptionally` e `handle`, fundamentais para controle de fluxo em tarefas não determinísticas.

A aula 7 abordou os Parallel Streams. Apresentei um exemplo ideal de uso, envolvendo uma tarefa CPU bound com grande volume de dados e estrutura de dados apropriada. Depois, mostrei dois contraexemplos: um com tarefas bloqueantes, onde o uso de Parallel Streams pode prejudicar o desempenho, e outro com operação não thread-safe, onde alterações concorrentes causam resultados incorretos. Reforcei a importância de utilizar coletores seguros para evitar esse tipo de problema.

Na aula 8, exploramos o poder das Virtual Threads. Mostrei como podemos criar centenas de milhares de threads simultaneamente, realizando tarefas leves como sleep, com desempenho excepcional. Fizemos uma comparação prática entre virtual threads e threads convencionais, destacando como o modelo de concorrência baseado em virtual threads simplifica o desenvolvimento e melhora a escalabilidade de aplicações modernas.

Essa aula de prática foi construída para amarrar todos os conceitos apresentados anteriormente. Cada exemplo reforça o conteúdo e permite que você experimente na prática o comportamento das soluções. Recomendo fortemente que você explore os códigos, modifique-os e observe o impacto de cada mudança. Esse processo ativo é essencial para consolidar o aprendizado e transformar o conhecimento teórico em habilidade prática.

